

দিন ৬

Tree ও Graph

Hierarchy আর network — সম্পর্কের ডেটা বোঝা। Recursion আর Queue-এর আসল প্রয়োগ এখানে

আজ যা শিখবো

- Tree-এর পরিভাষা ও Binary Tree
- Traversal: Inorder, Preorder, Postorder (DFS) ও Level Order (BFS)
- Binary Search Tree (BST) — search, insert, validate
- Graph: পরিভাষা ও representation (Adjacency List/Matrix)
- Graph BFS ও DFS, Connected Components
- Cycle Detection (Undirected) ও Bipartite Graph

কোর্স ভিডিও: ৩৩-৪০

দিন শেষে তুমি পারবে

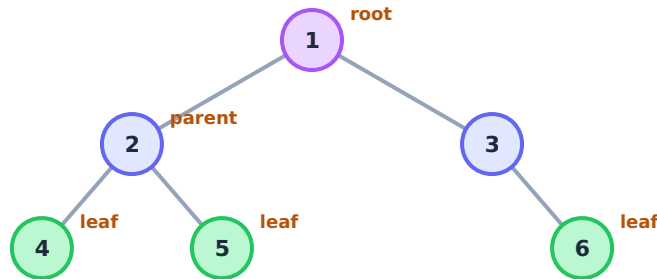
- Tree-এর ৪ ধরনের traversal লিখতে ও আউটপুট বলতে পারবে
- Max Depth আর Level Order নিজে লিখতে পারবে
- BST-এর নিয়ম আর validate করার সঠিক উপায় জানবে
- Graph-এ BFS/DFS চলিয়ে Number of Islands সলভ করতে পারবে

স্টাডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	Tree কনসেপ্ট + ৩টা traversal
১ ঘণ্টা	Max Depth + Level Order
৪৫ মিনিট	BST + Validate BST
১ ঘণ্টা	Graph কনসেপ্ট + BFS/DFS
১ ঘণ্টা	Islands + Components + Cycle + Bipartite
৩০ মিনিট	খিওরি + চিট শিট

অধ্যায় ১: Tree — গাছের মতো সাজানো ডেটা

Tree হলো node-এর একটা hierarchy (স্তরবিন্যাস)। সবার উপরে একটা **root**, প্রতিটা node-এর নিচে **child** থাকতে পারে। কোনো লুপ (cycle) নেই।



শব্দ	মানে	উপরের গাছে
Root	সবার উপরের node, কোনো parent নেই	1
Parent / Child	সরাসরি উপরের / নিচের node	2 হলো 4 আর 5-এর parent
Leaf	যার কোনো child নেই	4, 5, 6
Height	root থেকে সবচেয়ে দূরের leaf পর্যন্ত লেভেল সংখ্যা	3
Depth	root থেকে একটা node কত লেভেল নিচে	5-এর depth = 2 (root = 0)
Subtree	একটা node আর তার নিচের সবকিছু	2, 4, 5 মিলে একটা subtree

বাস্তব উদাহরণ

কম্পিউটারের ফোল্ডার স্ট্রাকচার, কোম্পানির org chart (CEO → ম্যানেজার → কর্মী), HTML DOM, Laravel-এর category-subcategory, বংশলতিকা।

Binary Tree

প্রতিটা node-এর সর্বোচ্চ দুইটা child: **left** আর **right**। ইন্টারভিউয়ের প্রায় সব tree প্রশ্ন binary tree নিয়ে।

PHP

```

class TreeNode {
    public $val;
    public ?TreeNode $left;
    public ?TreeNode $right;
    public function __construct($val = 0, ?TreeNode $left = null, ?TreeNode $right = null) {
        $this->val = $val;
        $this->left = $left;
        $this->right = $right;
    }
}

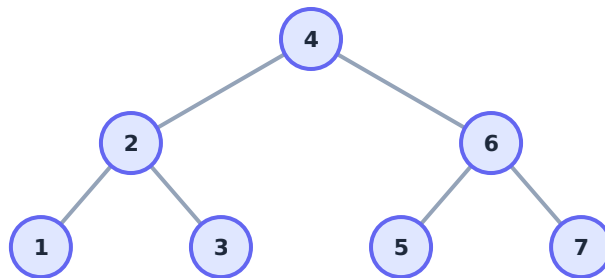
//      4
//     / \
//    2   6
$root = new TreeNode(4, new TreeNode(2), new TreeNode(6));
echo $root->left->val . " " . $root->right->val; // 2 6

```

এরপরের কোডগুলোর জন্য

ধরে নিচ্ছি `TreeNode` class আছে, আর `buildTree([4, 2, 6, 1, 3, 5, 7])` helper — LeetCode-এর মতো level-order array থেকে গাছ বানায় (`null` মানে সেখানে node নেই)। Helper-টা চিট শিটে দেওয়া আছে।

Traversal — গাছের প্রতিটা node ঘুরে দেখার 8 উপায়



নাম	ক্রম	আউটপুট	মনে রাখার কৌশল
Preorder (DFS)	Root → Left → Right	4, 2, 1, 3, 6, 5, 7	Root আগে (pre)। গাছ কপি করতে।
Inorder (DFS)	Left → Root → Right	1, 2, 3, 4, 5, 6, 7	Root মাঝে (in)। BST-তে sorted আসে!
Postorder (DFS)	Left → Right → Root	1, 3, 2, 5, 7, 6, 4	Root শেষে (post)। গাছ মুছতে।
Level Order (BFS)	লেভেল ধরে, বাম থেকে ডান	4, 2, 6, 1, 3, 5, 7	Queue ব্যবহার করে।

DFS বনাম BFS

DFS (Depth First): একটা পথে যতদূর যাওয়া যায় যাও, তারপর ফিরে আসো — recursion বা stack।

BFS (Breadth First): আগে কাছের সবাইকে দেখো (একই লেভেল), তারপর পরের লেভেল — queue।

প্রশ্ন 6.1

Binary Tree Traversals (Inorder, Preorder, Postorder)

গাছের তিনটা DFS traversal

Easy

LeetCode 94 / 144 / 145

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Tree-এর সব প্রশ্নের ভিত্তি। ইন্টারভিউতে প্রায়ই "inorder traversal recursive আর iterative দুইভাবে লেখো" বলে।

সমস্যাটা কী?

একটা binary tree-এর root দেওয়া আছে। Inorder, Preorder আর Postorder traversal-এর মানগুলো array হিসেবে রিটার্ন করো।

উদাহরণ

Input: [4, 2, 6, 1, 3, 5, 7]

Inorder: [1, 2, 3, 4, 5, 6, 7]

Preorder: [4, 2, 1, 3, 6, 5, 7]

Postorder: [1, 3, 2, 5, 7, 6, 4]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

তিনটা traversal-এর কোড প্রায় একই — শুধু "root-এর মান কখন result-এ যোগ করবো" লাইনটার জায়গা বদলায়: বাম-ডানের আগে (pre), মাঝে (in), নাকি পরে (post)।

পদ্ধতি ১: Recursive — তিনটাই একসাথে

1. Base: node null হলে ফিরে যাও
2. বাম subtree-তে যাও, ডান subtree-তে যাও
3. root-এর মান যোগ করার লাইনটা কোথায় বসছে সেটাই পার্থক্য

PHP

```

function inorder(?TreeNode $node, array &$res): void {
    if ($node === null) return;
    inorder($node->left, $res);
    $res[] = $node->val;          // মাঝে
    inorder($node->right, $res);
}

function preorder(?TreeNode $node, array &$res): void {
    if ($node === null) return;
    $res[] = $node->val;          // আগে
    preorder($node->left, $res);
    preorder($node->right, $res);
}

function postorder(?TreeNode $node, array &$res): void {
    if ($node === null) return;
    postorder($node->left, $res);
    postorder($node->right, $res);
    $res[] = $node->val;          // শেষে
}

$root = buildTree([4, 2, 6, 1, 3, 5, 7]);
$a = []; inorder($root, $a); echo implode(", ", $a) . "\n"; // 1,2,3,4,5,6,7
$b = []; preorder($root, $b); echo implode(", ", $b) . "\n"; // 4,2,1,3,6,5,7
$c = []; postorder($root, $c); echo implode(", ", $c); // 1,3,2,5,7,6,4

```

Time: $O(n)$ Space: $O(h)$ – h = গাছের উচ্চতা (recursion stack)

পদ্ধতি ২: Iterative Inorder — Stack দিয়ে (follow-up)

1. curr = root, একটা খালি stack
2. যতক্ষণ curr আছে: stack-এ push করে বামে যাও
3. curr null হলে: stack থেকে pop, মান যোগ করো, তারপর ডানে যাও

PHP

```

function inorderIterative(?TreeNode $root): array {
    $res = [];
    $stack = [];
    $curr = $root;
    while ($curr !== null || !empty($stack)) {
        while ($curr !== null) { // যতদূর সম্ভব বামে যাও
            $stack[] = $curr;
            $curr = $curr->left;
        }
        $curr = array_pop($stack); // সবচেয়ে বামের node
        $res[] = $curr->val;
        $curr = $curr->right; // এবার তার ডান subtree
    }
    return $res;
}

echo implode(", ", inorderIterative(buildTree([4, 2, 6, 1, 3, 5, 7]))); //
1,2,3,4,5,6,7

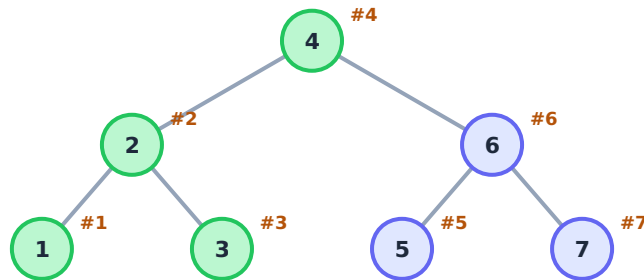
```

Time: $O(n)$ Space: $O(h)$

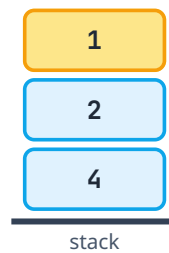
Recursion আসলে ভেতরে call stack ব্যবহার করে — এখানে আমরা সেই stack নিজের হাতে চালাচ্ছি।

ড্রাই রান — ধাপে ধাপে দেখি

Inorder ([4,2,6,1,3,5,7]) — কোন node কখন result-এ যায়:



ধাপ 1 4 → বামে 2 → বামে 1 → বামে null। stack = [4, 2, 1]



ধাপ 2 pop 1 → result [1]। 1-এর ডান null। pop 2 → [1, 2]। 2-এর ডানে 3



ধাপ 3 pop 3 → [1,2,3]। pop 4 → [1,2,3,4]। 4-এর ডানে 6 → বামে 5 ...



ধাপ 4 শেষে result = [1, 2, 3, 4, 5, 6, 7] — BST-তে inorder সবসময় sorted!



এজ কেস (এগুলো ভুলবে না)

- খালি গাছ (null) → []
- শুধু root → [root]
- একদিকে হেলে থাকা গাছ (linked list-এর মতো) → $h = n$, stack $O(n)$

ইন্টারভিউতে যা বলবে

- Space বলার সময় বলো $O(h)$: balanced গাছে $O(\log n)$, কিন্তু skewed গাছে $O(n)$ ।

প্রশ্ন 6.2

Maximum Depth of Binary Tree

গাছের সর্বোচ্চ গভীরতা (উচ্চতা)

Easy

LeetCode 104

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Tree recursion-এর সবচেয়ে পরিষ্কার উদাহরণ: "আমার উত্তর = বাচ্চাদের উত্তর থেকে তৈরি।" এই চিন্তা শত শত tree প্রশ্নে লাগে।

সমস্যাটা কী?

Binary tree-এর root থেকে সবচেয়ে দূরের leaf পর্যন্ত পথে কয়টা node — সেটা রিটার্ন করো।

উদাহরণ

Input: [3, 9, 20, null, null, 15, 7]

Output: 3

Input: []

Output: 0

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

কোম্পানির CEO জানতে চান org chart কত স্তর গভীর। তিনি দুই ভাইস-প্রেসিডেন্টকে জিজ্ঞেস করেন "তোমার নিচে কত স্তর?" তারা আবার তাদের নিচের জনদের জিজ্ঞেস করে... CEO দুইটা উত্তরের বড়টা নিয়ে নিজের জন্য ১ যোগ করেন।

পদ্ধতি ১: Recursive DFS (৩ লাইনের সমাধান)

1. Base: node null $\rightarrow$ 0
2. বাম আর ডান subtree-এর depth বের করো
3. উত্তর = $1 + \max(\text{বাম}, \text{ডান})$

PHP

```
function maxDepth(?TreeNode $root): int {
    if ($root === null) return 0;
    return 1 + max(maxDepth($root->left), maxDepth($root->right));
}
echo maxDepth(buildTree([3, 9, 20, null, null, 15, 7])); // 3
```

Time: $O(n)$ Space: $O(h)$

পদ্ধতি ২: BFS — লেভেল গোনো

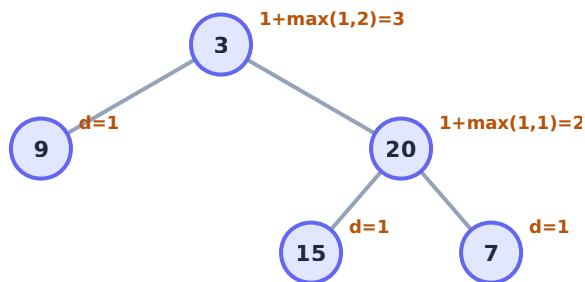
Level order-এ ঘুরো; প্রতিটা লেভেল শেষ হলে depth++।

PHP

```
function maxDepthBFS(?TreeNode $root): int {
    if ($root === null) return 0;
    $q = new SplQueue();
    $q->enqueue($root);
    $depth = 0;
    while (!$q->isEmpty()) {
        $depth++;
        $size = count($q); // এই লেভেলে কয়টা node
        for ($i = 0; $i < $size; $i++) {
            $node = $q->dequeue();
            if ($node->left) $q->enqueue($node->left);
            if ($node->right) $q->enqueue($node->right);
        }
    }
    return $depth;
}
echo maxDepthBFS(buildTree([3, 9, 20, null, null, 15, 7])); // 3
```

Time: $O(n)$ Space: $O(w)$ – w = সবচেয়ে চওড়া লেভেল

ড্রাই রান — ধাপে ধাপে দেখি



নিচ থেকে উপরে উত্তর ফেরে: leaf-গুলো 1 দেয়, 20 পায় $1 + \max(1,1) = 2$, root 3 পায় $1 + \max(1, 2) = 3$ ।

এজ কেস (এগুলো ভুলবে না)

- খালি গাছ $\rightarrow 0$
- শুধু root $\rightarrow 1$

ফালো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"Minimum depth?" (LeetCode 111) — সাবধান! একদিকের child null হলে সেই দিক leaf না, তাই শুধু অন্য দিকটা বিবেচনা করতে হবে। BFS-এ প্রথম যে leaf পাও সেটাই উত্তর।

প্রশ্ন 6.3

Binary Tree Level Order Traversal

লেভেল ধরে ধরে node গুলো (BFS)

Medium

LeetCode 102

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Tree-তে BFS-এর টেমপ্লেট। "প্রতিটা লেভেলের সবচেয়ে ডানের node", "zigzag", "লেভেলের গড়" — সব এই টেমপ্লেট থেকে।

সমস্যাটা কী?

Binary tree-এর মানগুলো লেভেল অনুযায়ী (বাম থেকে ডানে) আলাদা আলাদা list-এ রিটার্ন করো।

উদাহরণ

Input: [3, 9, 20, null, null, 15, 7]

Output: [[3], [9, 20], [15, 7]]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

স্কুলের অ্যাসেম্বলিতে সারি ধরে নাম ডাকা হচ্ছে: প্রথম সারি শেষ হলে দ্বিতীয় সারি। queue-তে এক সারির সবাই দাঁড়ায়, প্রত্যেকে বের হওয়ার সময় তার পেছনের (child) দের লাইনে দাঁড় করিয়ে যায়।

BFS with Queue — লেভেলের সাইজ ধরে

1. root-কে queue-তে রাখো
2. প্রতিটা লেভেলের শুরুতে queue-এর সাইজ = এই লেভেলের node সংখ্যা
3. ঠিক ততগুলো node বের করো, মান রাখো, তাদের child-দের queue-তে ঢোকাও
4. লেভেলের list result-এ যোগ করো

PHP

```
function levelOrder(?TreeNode $root): array {
    $result = [];
    if ($root === null) return $result;
    $q = new SplQueue();
    $q->enqueue($root);
    while (!$q->isEmpty()) {
        $level = [];
        $size = count($q); // এই লেভেলে কয়টা
        for ($i = 0; $i < $size; $i++) {
            $node = $q->dequeue();
            $level[] = $node->val;
            if ($node->left !== null) $q->enqueue($node->left);
            if ($node->right !== null) $q->enqueue($node->right);
        }
        $result[] = $level;
    }
    return $result;
}
echo json_encode(levelOrder(buildTree([3, 9, 20, null, null, 15, 7]))); // [[3], [9,20],[15,7]]
```

Time: $O(n)$ Space: $O(n)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 queue = [3], size = 1 → 3 বের করো, 9 আর 20 ঢোকাও। level = [3]

queue এখন

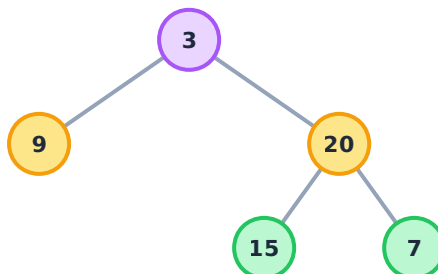


ধাপ 2 size = 2 → 9 বের (child নেই), 20 বের (15, 7 ঢোকাও)। level = [9, 20]

queue এখন



ধাপ 3 size = 2 → 15, 7 বের। level = [15, 7]। queue খালি → শেষ



এজ কেস (এগুলো ভুলবে না)

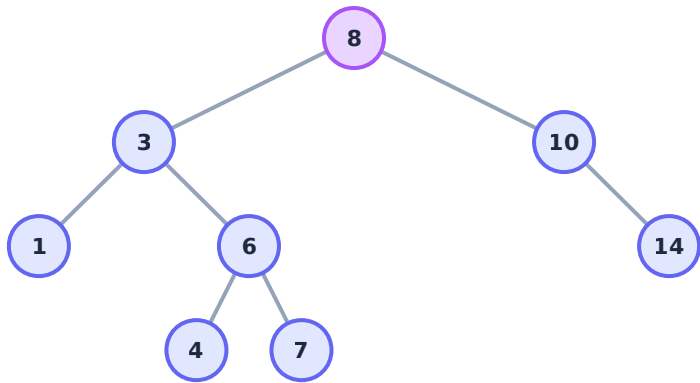
- খালি গাছ → []

ইন্টারভিউতে যা বলবে

- মূল কৌশল `$size = count($q)` লুপের আগেই নেওয়া — নইলে লুপের ভেতরে child ঢোকায় সাইজ বদলে যাবে আর লেভেল মিশে যাবে।

অধ্যায় ২: Binary Search Tree (BST)

BST হলো এমন binary tree যেখানে প্রতিটা node-এর জন্য: বাম subtree-এর সব মান ছোট, আর ডান subtree-এর সব মান বড়।



BST-এর জাদু

খোঁজার সময় প্রতিটা node-এ আমরা অর্ধেক গাছ বাদ দিই (binary search-এর মতো!)। 6 খুঁজছি: 8-এর চেয়ে ছোট → বামে; 3-এর চেয়ে বড় → ডানে; পেয়েছি 6।

অপারেশন	Balanced BST	Skewed BST (একদিকে হেলানো)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Skewed BST

1, 2, 3, 4, 5 ক্রমে insert করলে প্রতিটা ডানে বসে — গাছটা আসলে একটা linked list হয়ে যায়, উচ্চতা n । এজন্য বাস্তবে AVL বা Red-Black tree-এর মতো **self-balancing** BST ব্যবহার হয় (ইন্টারভিউতে নাম জানলেই যথেষ্ট)।

PHP — BST Search ও Insert

```

function searchBST(?TreeNode $node, int $target): ?TreeNode {
    while ($node !== null && $node->val !== $target) {
        $node = ($target < $node->val) ? $node->left : $node->right;    // অর্ধেক বাদ
    }
    return $node;
}

function insertBST(?TreeNode $node, int $val): TreeNode {
    if ($node === null) return new TreeNode($val);    // খালি জায়গা পেলে এখানেই বসো
    if ($val < $node->val) {
        $node->left = insertBST($node->left, $val);
    } else {
        $node->right = insertBST($node->right, $val);
    }
    return $node;
}

$root = null;
foreach ([8, 3, 10, 1, 6, 14, 4, 7] as $v) $root = insertBST($root, $v);
echo searchBST($root, 6) ? "6 পাওয়া গেছে" : "নেই";    // 6 পাওয়া গেছে
echo " | " . (searchBST($root, 5) ? "5 পাওয়া গেছে" : "5 নেই");    // 5 নেই

```

Time: O(h)

Space: O(1) search, O(h) insert (recursive)

প্রশ্ন 6.4

Validate Binary Search Tree

গাছটা সত্যিই BST কি না

Medium

LeetCode 98

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

BST-এর সংজ্ঞা সত্যি বুঝেছো কি না যাচাই করে। একটা বিখ্যাত ফাঁদ আছে — শুধু বাবা-বাচ্চা তুলনা করলে ভুল হয়।

সমস্যাটা কী?

Binary tree-এর root দেওয়া আছে। এটা valid BST কি না বলো (প্রতিটা node-এর বাম subtree-এর সব মান ছোট, ডান subtree-এর সব মান বড়)।

উদাহরণ

Input: [2, 1, 3]

Output: true

Input: [5, 4, 6, null, null, 3, 7]

Output: false // 3 আছে 5-এর ডান subtree-তে, কিন্তু $3 < 5$!

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

ফাঁদ: 6-এর বাম child 3 — 6-এর দৃষ্টিতে ঠিক আছে ($3 < 6$)। কিন্তু 3 পুরোটা 5-এর ডান পাশে, তাই 3-কে 5-এর চেয়ে বড় হতেই হবে। প্রতিটা node-কে শুধু বাবার না, **সব পূর্বপুরুষের** দেওয়া সীমা মানতে হয়।

পদ্ধতি ১: Inorder করে sorted কি না দেখা

BST-এর inorder traversal সবসময় কঠোরভাবে ক্রমবর্ধমান। তাই inorder array বানিয়ে চেক করো প্রতিটা আগেরটার চেয়ে বড় কি না।

PHP

```
function collectInorder(?TreeNode $n, array &$out): void {
    if ($n === null) return;
    collectInorder($n->left, $out);
    $out[] = $n->val;
    collectInorder($n->right, $out);
}

function isValidBSTInorder(?TreeNode $root): bool {
    $vals = [];
    collectInorder($root, $vals);
    for ($i = 1; $i < count($vals); $i++) {
        if ($vals[$i] <= $vals[$i - 1]) return false;
    }
    return true;
}

var_dump(isValidBSTInorder(buildTree([5, 4, 6, null, null, 3, 7]))); // false
```

Time: $O(n)$ Space: $O(n)$

পদ্ধতি ২: সীমা (min, max) নিয়ে Recursion

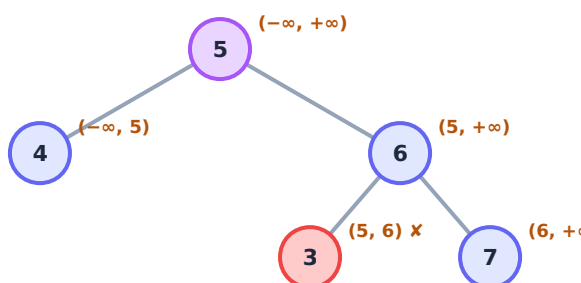
- প্রতিটা node-এর একটা বৈধ রেঞ্জ (min, max) আছে
- বামে গেলে $\text{max} = \text{node-এর মান}$; ডানে গেলে $\text{min} = \text{node-এর মান}$
- কোনো node রেঞ্জের বাইরে গেলেই false

PHP

```
function isValidBST(?TreeNode $node, ?int $min = null, ?int $max = null): bool {
    if ($node === null) return true;
    if (($min !== null && $node->val <= $min) || ($max !== null && $node->val >= $max))
    {
        return false; // সীমার বাইরে
    }
    return isValidBST($node->left, $min, $node->val) // বামে: উপরের সীমা কমে
        && isValidBST($node->right, $node->val, $max); // ডানে: নিচের সীমা বাড়ে
}
var_dump(isValidBST(buildTree([2, 1, 3]))); // true
var_dump(isValidBST(buildTree([5, 4, 6, null, null, 3, 7]))); // false
```

Time: $O(n)$ Space: $O(h)$

ড্রাই রান — ধাপে ধাপে দেখি



প্রতিটা node-এর পাশে তার বৈধ রেঞ্জ। 3-এর রেঞ্জ (5, 6) — কিন্তু $3 \leq 5$, তাই false।

এজ কেস (এগুলো ভুলবে না)

- খালি গাছ $\rightarrow$ true
- ডুপ্লিকেট মান $\rightarrow$ সাধারণত invalid (প্রশ্ন করে নিশ্চিত হও)
- PHP_INT_MIN/MAX-এর মান থাকতে পারে — তাই null দিয়ে "সীমা নেই" বোঝানো নিরাপদ

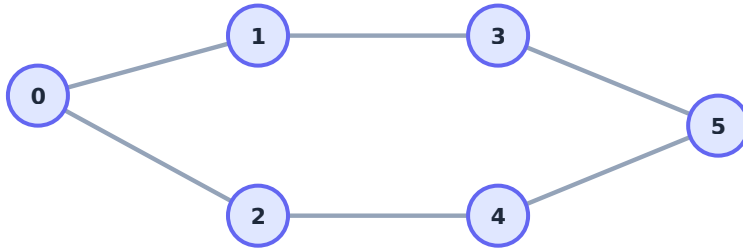
ইন্টারভিউতে যা বলবে

- প্রথমে ভুল পদ্ধতিটা (শুধু child-এর সাথে তুলনা) নিজে উল্লেখ করে counter-example দেখাও — ইন্টারভিউয়ার বুঝবে তুমি ফাঁদটা জানো।

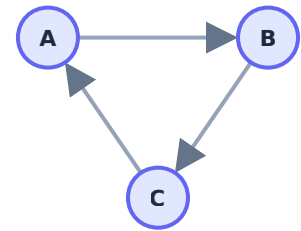
অধ্যায় ৩: Graph

Graph হলো **vertex** (node) আর তাদের মধ্যে **edge** (সংযোগ)-এর সংগ্রহ। Tree আসলে একটা বিশেষ graph (cycle নেই, সব সংযুক্ত)। Graph-এ cycle থাকতে পারে, root নেই।

Undirected graph (দুই দিকে যাওয়া যায়)



Directed graph (একমুখী)



ধরন	মানে	উদাহরণ
Undirected	edge দুই দিকে	Facebook friend (A বন্ধু B → B বন্ধু A)
Directed	edge এক দিকে	Twitter follow, course prerequisite
Weighted	edge-এর একটা মান (দূরত্ব/খরচ)	Google Maps-এ রাস্তা (কিমি)
Connected	যেকোনো node থেকে যেকোনো node-এ যাওয়া যায়	—
Cycle	কোনো node থেকে বেরিয়ে আবার সেখানে ফেরার পথ	A → B → C → A

Graph কীভাবে রাখবো? (Representation)

Adjacency List (প্রতিটা node → তার প্রতিবেশীরা)

node	প্রতিবেশী
0	1, 2
1	0, 3
2	0, 4
3	1, 5
4	2, 5
5	3, 4

Adjacency Matrix ($\text{matrix}[i][j] = 1$ মানে edge আছে)

0	1	1	0	0	0
1	0	0	1	0	0
1	0	0	0	1	0
0	1	0	0	0	1
0	0	1	0	0	1
0	0	0	1	1	0

	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
দুইটা node-এর মধ্যে edge আছে?	$O(\text{প্রতিবেশী সংখ্যা})$	$O(1)$
সব প্রতিবেশী ঘোরা	$O(\text{প্রতিবেশী সংখ্যা})$	$O(V)$
কখন	বেশিরভাগ ক্ষেত্রে (sparse graph)	ঘন graph, ছোট V

PHP

```
// edge list থেকে adjacency list বানানো (undirected)
function buildGraph(int $n, array $edges): array {
    $adj = array_fill(0, $n, []);
    foreach ($edges as [$u, $v]) {
        $adj[$u][] = $v;
        $adj[$v][] = $u;          // undirected, তাই দুই দিকেই
    }
    return $adj;
}

$adj = buildGraph(6, [[0,1], [0,2], [1,3], [2,4], [3,5], [4,5]]);
echo json_encode($adj);    // [[1,2],[0,3],[0,4],[1,5],[2,5],[3,4]]
```

প্রশ্ন 6.5

Graph Traversal: BFS ও DFS

Graph-এর সব node ঘুরে দেখা

Easy

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

সব graph প্রশ্নের ভিত্তি। Tree-র সাথে পার্থক্য একটাই: cycle থাকতে পারে, তাই **visited** রাখতেই হবে — নইলে অসীম লুপ।

সমস্যাটা কী?

Adjacency list আর একটা start node দেওয়া আছে। BFS আর DFS দুইভাবে সব reachable node কোন ক্রমে ভিজিট হয় রিটার্ন করো।

উদাহরণ

Graph: 0-1, 0-2, 1-3, 2-4, 3-5, 4-5, start = 0

BFS: [0, 1, 2, 3, 4, 5]

DFS: [0, 1, 3, 5, 4, 2]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

BFS = পুকুরে ঢিল ছুড়লে ঢেউ যেমন চারদিকে সমানভাবে ছড়ায় — আগে কাছের সবাই, তারপর দূরের। **DFS** = গোলকধাঁধায় একটা পথ ধরে শেষ পর্যন্ত যাও, দেয়ালে ঠেকলে পেছনে ফিরে অন্য পথ ধরো। দুটোতেই দাগ দিয়ে রাখো কোথায় আগে গেছো (visited)।

BFS — Queue দিয়ে

1. start-কে visited করে queue-তে রাখো
2. queue থেকে একটা বের করো, result-এ রাখো
3. তার প্রতিটা unvisited প্রতিবেশীকে visited করে queue-তে ঢোকাও

PHP

```
function bfs(array $adj, int $start): array {
    $visited = [$start => true];
    $order = [];
    $q = new SplQueue();
    $q->enqueue($start);
    while (!$q->isEmpty()) {
        $node = $q->dequeue();
        $order[] = $node;
        foreach ($adj[$node] as $nb) {
            if (!isset($visited[$nb])) {
                $visited[$nb] = true; // ঢোকানোর সময়ই দাগ দাও
                $q->enqueue($nb);
            }
        }
    }
    return $order;
}

$adj = [[1, 2], [0, 3], [0, 4], [1, 5], [2, 5], [3, 4]];
echo implode(" ", bfs($adj, 0)); // 0, 1, 2, 3, 4, 5
```

Time: $O(V + E)$ Space: $O(V)$

DFS — Recursion দিয়ে

1. node-কে visited করো, result-এ রাখো
2. প্রতিটা unvisited প্রতিবেশীর জন্য DFS কল করো

PHP

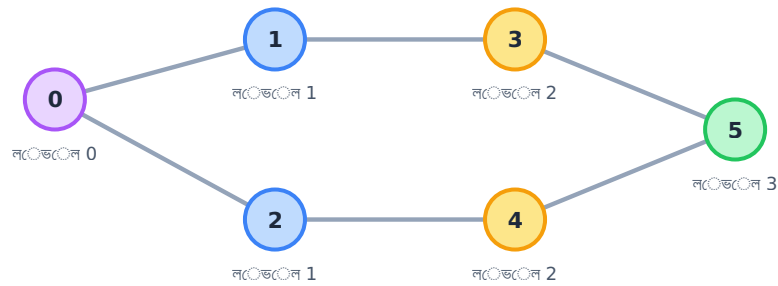
```
function dfs(array $adj, int $node, array &$amp;visited, array &$amp;order): void {
    $visited[$node] = true;
    $order[] = $node;
    foreach ($adj[$node] as $nb) {
        if (!isset($visited[$nb])) {
            dfs($adj, $nb, $visited, $order); // যতদূর সম্ভব গভীরে
        }
    }
}

$adj = [[1, 2], [0, 3], [0, 4], [1, 5], [2, 5], [3, 4]];
$visited = []; $order = [];
dfs($adj, 0, $visited, $order);
echo implode(" ", $order); // 0, 1, 3, 5, 4, 2
```

Time: $O(V + E)$ Space: $O(V)$

ড্রাই রান — ধাপে ধাপে দেখি

BFS (লেভেল অনুযায়ী ছড়ায়):



ধাপ	বের করলাম	queue-তে ঢোকালাম	queue
১	0	1, 2	[1, 2]
২	1	3	[2, 3]
৩	2	4	[3, 4]
৪	3	5	[4, 5]
৫	4	— (5 আগেই visited)	[5]
৬	5	—	[]

DFS: 0 → 1 → 3 → 5 → 4 → 2 → (4-এর সব প্রতিবেশী visited, ফেরত) ... শেষ

এজ কেস (এগুলো ভুলবে না)

- Disconnected graph → start থেকে সব node পৌঁছানো যায় না; সব node-এর জন্য লুপ চালাতে হয় (connected components দেখো)
- Self loop / multiple edge → visited থাকায় সমস্যা নেই

ইন্টারভিউতে যা বলবে

- BFS unweighted graph-এ **সবচেয়ে ছোট পথ** (কম edge) দেয় — এটা বললে ভালো। DFS path খোঁজা, cycle, backtracking-এ ভালো।
- BFS-এ queue-তে ঢোকানোর সময়ই visited করো, বের করার সময় না — নইলে একই node একাধিকবার queue-তে ঢুকতে পারে।

প্রশ্ন 6.6

Number of Islands

গ্রিডে কয়টা দ্বীপ

Medium

LeetCode 200

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Graph প্রশ্নের সবচেয়ে জনপ্রিয় রূপ — grid আসলে লুকানো graph (প্রতিটা ঘর node, পাশের ঘর edge)। প্রায় সব কোম্পানি জিজ্ঞেস করে।

সমস্যাটা কী?

"1" (মাটি) আর "0" (পানি)-এর একটা 2D grid। উপর-নিচ-বাম-ডানে লাগোয়া মাটি মিলে একটা দ্বীপ। মোট দ্বীপ সংখ্যা বের করো।

উদাহরণ

Input:

```
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
```

Output: 3

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

তুমি প্লেন থেকে ম্যাপ দেখছো। প্রথম যে মাটি চোখে পড়লো, সেখানে লাল রং ঢালো — রংটা পুরো দ্বীপে ছড়িয়ে যাবে (DFS)। গোনো ১। তারপর রং-না-লাগা পরের মাটি খোঁজো। মোট কতবার রং ঢাললে, সেটাই দ্বীপ সংখ্যা।

DFS দিয়ে "ডুবিয়ে দাও" (Flood Fill)

1. পুরো grid ঘুরো
2. কোনো ঘরে "1" পেলে: count++ আর সেখান থেকে DFS চালাও
3. DFS: grid-এর বাইরে বা "0" হলে ফিরে যাও; নইলে ঘরটা "0" বানাও (visited) আর চার দিকে DFS
4. এভাবে পুরো দ্বীপ একবারেই মুছে যায়, আবার গোনো হয় না

PHP

```

function numIslands(array $grid): int {
    $rows = count($grid);
    $cols = count($grid[0]);
    $count = 0;
    for ($r = 0; $r < $rows; $r++) {
        for ($c = 0; $c < $cols; $c++) {
            if ($grid[$r][$c] === '1') {
                $count++;
                sink($grid, $r, $c);
            }
        }
    }
    return $count;
}

function sink(array &$amp;grid, int $r, int $c): void {
    if ($r < 0 || $c < 0 || $r >= count($grid) || $c >= count($grid[0]) || $grid[$r]
[$c] !== '1') {
        return;
    }
    $grid[$r][$c] = '0';
    sink($grid, $r + 1, $c);
    sink($grid, $r - 1, $c);
    sink($grid, $r, $c + 1);
    sink($grid, $r, $c - 1);
}

$grid = [
    ['1', '1', '0', '0', '0'],
    ['1', '1', '0', '0', '0'],
    ['0', '0', '1', '0', '0'],
    ['0', '0', '0', '1', '1']
];
echo numIslands($grid); // 3

```

Time: $O(\text{rows} \times \text{cols})$ Space: $O(\text{rows} \times \text{cols})$ – সবচেয়ে খারাপে recursion-এর গভীরতা

ড্রাই রান – ধাপে ধাপে দেখি

তিনটা দ্বীপ তিন রঙে

1	1	0	0	0
1	1	0	0	0
0	0	1	0	0
0	0	0	1	1

1. (0,0)-এ "1" → count = 1। DFS নীল দ্বীপের ৪টা ঘর "0" করে দেয়।
2. (2,2)-এ "1" → count = 2। হলুদ দ্বীপ (১ ঘর)।
3. (3,3)-এ "1" → count = 3। সবুজ দ্বীপ (২ ঘর)।
4. আর কোনো "1" নেই → উত্তর 3।

এজ কেস (এগুলো ভুলবে না)

- সব পানি $\rightarrow 0$
- সব মাটি $\rightarrow 1$
- কোণাকুণি লাগোয়া মাটি আলাদা দ্বীপ (শুধু 8 দিক গোনো হয়)

ইন্টারভিউতে যা বলবে

- ইনপুট grid বদলে ফেলা ঠিক আছে কি না জিজ্ঞেস করো। না হলে আলাদা visited array রাখো।
- খুব বড় grid-এ recursion গভীর হয় — BFS (queue) দিয়েও একই সমাধান করা যায়।

প্রশ্ন 6.7

Number of Connected Components

Graph-এ কয়টা আলাদা অংশ

Medium

LeetCode 323

কেন এটা গুরুত্বপূর্ণ?

Number of Islands-এর graph ভাঙ্গন। "কয়টা ব্লুদের দল", "কয়টা আলাদা নেটওয়ার্ক" — এই ধরনের প্রশ্ন।

সমস্যাটা কী?

n টা node (0 থেকে $n-1$) আর undirected edge-এর list দেওয়া আছে। Connected component (এমন দল যার ভেতরে সবাই সবার কাছে পৌঁছাতে পারে) কয়টা?

উদাহরণ

Input: $n = 6$, $edges = [[0,1], [1,2], [3,4]]$
 Output: 3 // $\{0,1,2\}$, $\{3,4\}$, $\{5\}$

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একটা শহরে কিছু বাড়ি রাস্তা দিয়ে যুক্ত। কয়টা আলাদা "পাড়া" আছে যাদের মধ্যে কোনো রাস্তা নেই? প্রতিটা নতুন (অদেখা) বাড়ি থেকে পুরো পাড়া ঘুরে দাগ দাও, আর গোনো।

প্রতিটা unvisited node থেকে DFS

1. adjacency list বানাও
2. সব node-এর উপর লুপ: unvisited হলে count++ আর DFS দিয়ে তার পুরো component visited করো

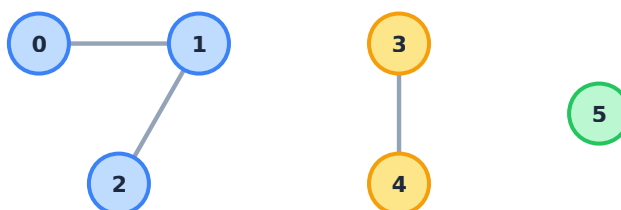
PHP

```
function countComponents(int $n, array $edges): int {
    $adj = array_fill(0, $n, []);
    foreach ($edges as [$u, $v]) { $adj[$u][] = $v; $adj[$v][] = $u; }
    $visited = [];
    $count = 0;
    for ($i = 0; $i < $n; $i++) {
        if (!isset($visited[$i])) {
            $count++; // নতুন component
            $stack = [$i]; // iterative DFS
            $visited[$i] = true;
            while (!empty($stack)) {
                $node = array_pop($stack);
                foreach ($adj[$node] as $nb) {
                    if (!isset($visited[$nb])) { $visited[$nb] = true; $stack[] = $nb; }
                }
            }
        }
    }
    return $count;
}
echo countComponents(6, [[0, 1], [1, 2], [3, 4]]); // 3
```

Time: $O(V + E)$ Space: $O(V + E)$

ড্রাই রান — ধাপে ধাপে দেখি

তিনটা component তিন রঙে: {0,1,2}, {3,4}, {5}



এজ কেস (এগুলো ভুলবে না)

- কোনো edge নেই $\rightarrow$ n টা component
- সব যুক্ত $\rightarrow$ 1

ইন্টারভিউতে যা বলবে

- বিকল্প: **Union-Find (Disjoint Set)** — নাম জানলে উল্লেখ করো; শুরুতে n component, প্রতিটা নতুন সংযোগে ১ কমে।

প্রশ্ন 6.8

Detect Cycle in an Undirected Graph

Undirected graph-এ cycle আছে কি না

Medium

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

কোর্সের ভিডিও ৩৯-এর বিষয়। "parent ট্রাক করা" কৌশল — undirected graph-এর বিশেষ চিন্তা শেখায়।

সমস্যাটা কী?

n টা node আর undirected edge দেওয়া আছে। Graph-এ কোনো cycle থাকলে true।

উদাহরণ

Input: n = 4, edges = [[0,1], [1,2], [2,0], [2,3]]

Output: true // 0-1-2-0

Input: n = 4, edges = [[0,1], [1,2], [1,3]]

Output: false

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

গোলকধাঁধায় হাঁটার সময় তুমি যেখান থেকে এসেছো (parent), সেখানে ফেরাটা স্বাভাবিক — ওটা cycle না। কিন্তু যদি অন্য কোনো পথে এমন জায়গায় পৌঁছাও যেখানে আগেই দাগ দিয়েছো, তাহলে নিশ্চয়ই একটা বৃত্তাকার পথ আছে!

DFS + parent

1. প্রতিটা unvisited node থেকে DFS(node, parent)
2. প্রতিবেশী unvisited → DFS(প্রতিবেশী, node)
3. প্রতিবেশী visited এবং সে parent না → cycle পাওয়া গেছে!

PHP

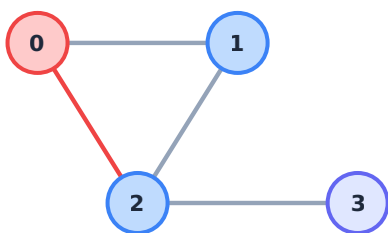
```
function hasCycleUndirected(int $n, array $edges): bool {
    $adj = array_fill(0, $n, []);
    foreach ($edges as [$u, $v]) { $adj[$u][] = $v; $adj[$v][] = $u; }
    $visited = [];
    for ($i = 0; $i < $n; $i++) { // disconnected graph-এর জন্য
        if (!isset($visited[$i]) && dfsCycle($adj, $i, -1, $visited)) {
            return true;
        }
    }
    return false;
}

function dfsCycle(array $adj, int $node, int $parent, array &$visited): bool {
    $visited[$node] = true;
    foreach ($adj[$node] as $nb) {
        if (!isset($visited[$nb])) {
            if (dfsCycle($adj, $nb, $node, $visited)) return true;
        } elseif ($nb !== $parent) {
            return true; // visited, কিন্তু parent না → cycle
        }
    }
    return false;
}

var_dump(hasCycleUndirected(4, [[0, 1], [1, 2], [2, 0], [2, 3]])); // true
var_dump(hasCycleUndirected(4, [[0, 1], [1, 2], [1, 3]])); // false
```

Time: $O(V + E)$ Space: $O(V)$

ড্রাই রান — ধাপে ধাপে দেখি



1. DFS(0, parent -1): 0 visited → প্রতিবেশী 1
2. DFS(1, parent 0): প্রতিবেশী 0 = parent, skip; 2 unvisited
3. DFS(2, parent 1): প্রতিবেশী 1 = parent skip; প্রতিবেশী 0 visited এবং parent না → cycle!

এজ কেস (এগুলো ভুলবে না)

- Disconnected graph → প্রতিটা component-এ আলাদা DFS
- $n = 1$, কোনো edge নেই → false

ইন্টারভিউতে যা বলবে

- **Directed** graph-এ এই কৌশল চলে না! সেখানে "বর্তমান recursion path-এ আছে কি না" (৩ রঙ: white/gray/black) চেক করতে হয় — Course Schedule (207) প্রশ্নে লাগে।

প্রশ্ন 6.9

Is Graph Bipartite?

দুই দলে ভাগ করা যায় কি না

Medium

LeetCode 785

কেন এটা গুরুত্বপূর্ণ?

কোর্সের শেষ ভিডিওর বিষয়। BFS + রং করা (2-coloring) — "দুই দলে ভাগ করো যাতে একই দলের কেউ পরস্পর যুক্ত না থাকে" ধরনের সমস্যা।

সমস্যাটা কী?

Graph-এর node গুলোকে দুইটা দলে ভাগ করা যায় কি এমনভাবে যে প্রতিটা edge এক দলের node-কে অন্য দলের node-এর সাথে যুক্ত করে? (adjacency list দেওয়া)

উদাহরণ

Input: `[[1,3], [0,2], [1,3], [0,2]]` // বর্গ: 0-1-2-3-0

Output: `true` // দল A: {0, 2}, দল B: {1, 3}

Input: `[[1,2], [0,2], [0,1]]` // ত্রিভুজ

Output: `false`

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একটা পার্টিতে কিছু লোকের মধ্যে ঝগড়া আছে (edge)। তাদের দুইটা রুমে এমনভাবে পাঠাতে চাও যাতে ঝগড়া করা দুজন কখনো একই রুমে না থাকে। একজনকে লাল রুমে দাও, তার সব শত্রুকে নীল রুমে, তাদের শত্রুদের লাল রুমে... কোথাও দুই শত্রু একই রঙ পেলে — অসম্ভব।

BFS দিয়ে দুই রঙে রং করা

1. color array: -1 = এখনো রং হয়নি
2. প্রতিটা রং-না-হওয়া node থেকে BFS: তাকে 0 রং দাও
3. প্রতিবেশী রং-না-হওয়া → উল্টো রং (1 - আমার রং) দাও, queue-তে রাখো
4. প্রতিবেশীর রং আমার রঙের সমান → false

PHP

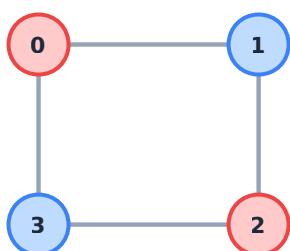
```
function isBipartite(array $graph): bool {
    $n = count($graph);
    $color = array_fill(0, $n, -1);
    for ($start = 0; $start < $n; $start++) {
        if ($color[$start] !== -1) continue;
        $color[$start] = 0;
        $q = new SplQueue();
        $q->enqueue($start);
        while (!$q->isEmpty()) {
            $node = $q->dequeue();
            foreach ($graph[$node] as $nb) {
                if ($color[$nb] === -1) {
                    $color[$nb] = 1 - $color[$node]; // উল্টো রং
                    $q->enqueue($nb);
                } elseif ($color[$nb] === $color[$node]) {
                    return false; // দুই প্রতিবেশী একই রং!
                }
            }
        }
    }
    return true;
}

var_dump(isBipartite([[1, 3], [0, 2], [1, 3], [0, 2]])); // true
var_dump(isBipartite([[1, 2], [0, 2], [0, 1]])); // false
```

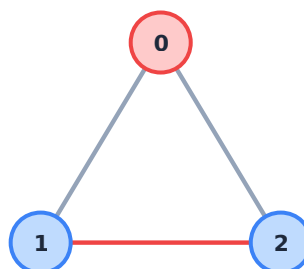
Time: $O(V + E)$ Space: $O(V)$

ড্রাই রান — ধাপে ধাপে দেখি

বর্গ: লাল {0,2}, নীল {1,3} → true



ত্রিভুজ: 1 আর 2 দুজনেই নীল হতে চায় → false



এজ কেস (এগুলো ভুলবে না)

- Disconnected graph → প্রতিটা component আলাদা রং করো (বাইরের লুপ এজন্যই)
- কোনো edge নেই → true

ইন্টারভিউতে যা বলবে

- মনে রাখার মতো তথ্য: graph bipartite হবে যদি এবং কেবল যদি এতে কোনো **বিজোড় দৈর্ঘ্যের cycle** না থাকে (ত্রিভুজ = ৩, বিজোড়)।

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Tree আর Graph-এর পার্থক্য কী?

Tree একটা বিশেষ graph: সংযুক্ত, কোনো cycle নেই, একটা root আছে, n node-এ ঠিক $n-1$ edge, প্রতিটা node-এর (root ছাড়া) একটাই parent। Graph-এ cycle থাকতে পারে, একাধিক অংশ থাকতে পারে, root-এর ধারণা নেই — তাই traversal-এ visited রাখতে হয়।

প্রশ্ন 2: Binary Tree আর Binary Search Tree-এর পার্থক্য?

Binary tree-তে প্রতিটা node-এর সর্বোচ্চ দুই child — মানের কোনো নিয়ম নেই। BST-তে অতিরিক্ত নিয়ম: প্রতিটা node-এর বাম subtree-এর সব মান ছোট, ডান subtree-এর সব মান বড়। এতে search/insert $O(h)$ হয়।

প্রশ্ন 3: BST-তে search-এর complexity কত? কখন খারাপ হয়?

$O(h)$, h = উচ্চতা। Balanced হলে $h = \log n$, তাই $O(\log n)$ । কিন্তু sorted ক্রমে insert করলে গাছ একদিকে হেলে linked list-এর মতো হয়, $h = n \rightarrow O(n)$ । সমাধান: AVL, Red-Black-এর মতো self-balancing tree।

প্রশ্ন 4: Inorder, Preorder, Postorder — পার্থক্য আর ব্যবহার?

Root কখন ভিজিট হয় সেটাই পার্থক্য। Preorder (Root-L-R): গাছ কপি বা serialize করতে। Inorder (L-Root-R): BST থেকে sorted ক্রমে মান পেতে। Postorder (L-R-Root): গাছ মুছতে বা বাচ্চাদের উত্তর থেকে বাবার উত্তর বের করতে (যেমন উচ্চতা, ফোল্ডারের মোট সাইজ)।

প্রশ্ন 5: BFS আর DFS-এর পার্থক্য? কখন কোনটা?

	BFS	DFS
ডেটা স্ট্রাকচার	Queue	Stack / Recursion
চলার ধরন	লেভেল ধরে, কাছ থেকে দূরে	এক পথে গভীরে, তারপর ফেরত
সেবা ব্যবহার	Unweighted graph-এ shortest path, level order	Path খোঁজা, cycle, connected component, backtracking
Time	$O(V + E)$	$O(V + E)$

প্রশ্ন 6: Adjacency List আর Matrix — কোনটা কখন?

List: space $O(V+E)$, বেশিরভাগ বাস্তব graph (sparse) এর জন্য সেবা, প্রতিবেশী ঘোরা দ্রুত। Matrix: space $O(V^2)$, দুইটা node-এর মধ্যে edge আছে কি না $O(1)$ -এ জানা যায় — ছোট বা ঘন (dense) graph-এ ভালো।

প্রশ্ন 7: Graph traversal-এ visited কেন লাগে?

Graph-এ cycle থাকতে পারে। visited না রাখলে $A \rightarrow B \rightarrow A \rightarrow B \dots$ অসীম লুপে পড়বে, আর একই node একাধিকবার প্রসেস হবে।

প্রশ্ন 8: Undirected graph-এ cycle কীভাবে খুঁজবে?

DFS চালাই আর প্রতিটা node-এর parent মনে রাখি। কোনো প্রতিবেশী যদি আগেই visited হয় এবং সে আমার parent না হয়, তাহলে cycle আছে। (BFS দিয়েও একইভাবে করা যায়, বা Union-Find দিয়ে।)

প্রশ্ন 9: Bipartite graph কী?

যে graph-এর node গুলো দুই দলে ভাগ করা যায় যাতে প্রতিটা edge দুই আলাদা দলের মধ্যে থাকে। BFS/DFS দিয়ে দুই রঙে রং করে চেক করি — কোনো edge-এর দুই মাথা একই রং পেলে bipartite না। বিজোড় দৈর্ঘ্যের cycle থাকলে কখনো bipartite হয় না।

প্রশ্ন 10: Connected component কী আর কীভাবে গুনবে?

এমন node-এর দল যাদের ভেতরে যেকোনো দুজনের মধ্যে পথ আছে, কিন্তু অন্য দলের সাথে কোনো সংযোগ নেই। সব node-এর উপর লুপ চালাই; কোনো unvisited node পেলে count++ করে তার থেকে BFS/DFS দিয়ে পুরো দল visited করি।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. Invert Binary Tree (Easy · LeetCode 226)

গাছের আয়না-প্রতিবিম্ব বানাও (প্রতিটা node-এর বাম-ডান অদলবদল)।

হিন্ট: Recursion: বাম আর ডান swap করো, তারপর দুই পাশে একই কাজ।

2. Same Tree (Easy · LeetCode 100)

দুইটা গাছ ছবছ এক কি না।

হিন্ট: দুটোই null $\rightarrow$ true; একটা null $\rightarrow$ false; মান আলাদা $\rightarrow$ false; নইলে বাম-বাম আর ডান-ডান recursive চেক।

3. Lowest Common Ancestor of a BST (Medium · LeetCode 235)

BST-তে দুইটা node-এর সবচেয়ে কাছের সাধারণ পূর্বপুরুষ।

হিন্ট: দুটোই ছোট হলে বামে, দুটোই বড় হলে ডানে; নইলে এই node-ই উত্তর।

4. Kth Smallest Element in a BST (Medium · LeetCode 230)

BST-এর k-তম ছোট মান।

হিন্ট: Inorder traversal sorted দেয় — k-তম ভিজিট করা node-ই উত্তর।

5. Flood Fill (Easy · LeetCode 733)

Paint-এর bucket tool: একটা ঘর থেকে একই রঙের সব লাগোয়া ঘর নতুন রঙ করো।

হিন্ট: Number of Islands-এর sink()-এর মতোই DFS।

6. Rotting Oranges (Medium · LeetCode 994)

পচা কমলা প্রতি মিনিটে পাশেরগুলোকে পচায়। সব পচতে কত মিনিট?

হিন্ট: Multi-source BFS: শুরুতে সব পচা কমলা queue-তে; প্রতিটা লেভেল = ১ মিনিট।

7. Course Schedule (Medium · LeetCode 207)

Prerequisite মেনে সব কোর্স শেষ করা সম্ভব কি না।

হিন্ট: Directed graph-এ cycle খোঁজা। Kahn's algorithm (in-degree 0 থেকে BFS) বা ৩-রঙের DFS।

চিট শিট — দিন ৬

Tree টেমপ্লেট

কাজ	মূল কোড	Time / Space
Max Depth	<code>1 + max(depth(L), depth(R))</code>	$O(n) / O(h)$
Inorder	<code>L → \$res[] = val → R</code>	$O(n) / O(h)$
Level Order	<code>queue + \$size = count(\$q)</code>	$O(n) / O(n)$
BST Search	ছোট হলে বামে, বড় হলে ডানে	$O(h) / O(1)$
Validate BST	recursion-এ (min, max) সীমা পাঠাও	$O(n) / O(h)$

Graph টেমপ্লেট

কাজ	মূল কৌশল	Time
BFS	SplQueue + visited (ডোকানোর সময় দাগ)	$O(V+E)$
DFS	recursion/stack + visited	$O(V+E)$
Grid (Islands)	৪ দিকে DFS, ঘর "0" করে visited	$O(R \times C)$
Components	প্রতিটা unvisited node থেকে DFS, count++	$O(V+E)$
Cycle (undirected)	visited প্রতিবেশী $\neq$ parent $\rightarrow$ cycle	$O(V+E)$
Bipartite	BFS-এ ০/১ রং, একই রং প্রতিবেশী $\rightarrow$ false	$O(V+E)$

buildTree helper (level-order array থেকে গাছ)

PHP — helper

```
function buildTree(array $vals): ?TreeNode {
    if (empty($vals) || $vals[0] === null) return null;
    $root = new TreeNode($vals[0]);
    $q = new SplQueue(); $q->enqueue($root); $i = 1;
    while (!$q->isEmpty() && $i < count($vals)) {
        $node = $q->dequeue();
        if ($i < count($vals) && $vals[$i] !== null) { $node->left = new
TreeNode($vals[$i]); $q->enqueue($node->left); }
        $i++;
        if ($i < count($vals) && $vals[$i] !== null) { $node->right = new
TreeNode($vals[$i]); $q->enqueue($node->right); }
        $i++;
    }
    return $root;
}
```

`echo buildTree([1, 2, 3])->right->val; // 3`

আজকের নিজেকে যাচাই

- ☐ একটা গাছ দেখে ৪টা traversal-এর আউটপুট বলতে পারি
- ☐ Max Depth আর Level Order না দেখে লিখতে পারি
- ☐ Validate BST-এর ফাঁদটা ব্যাখ্যা করতে পারি
- ☐ BFS vs DFS তুলনা করতে পারি
- ☐ Number of Islands লিখতে পারি